

PROJECT ROAR

Adaptive Multi-Agent Prompt Engineering Intelligent Tutoring System — Comprehensive Technical Report

System Name: Project ROAR v2.0	Backend Engine: FastAPI / Python 3.14
Inference Layer: Ollama Cloud / Local / OpenRouter	Knowledge Graph: 36 Nodes across 3 Difficulty Tiers
Vector Store: ChromaDB (Semantic RAG)	Database: SQLite + SQLAlchemy ORM

1. Executive Summary & Pedagogical Objective

Project ROAR is an academic-grade Intelligent Tutoring System (ITS) engineered to educate students, software engineers, and researchers in the systematic principles of Prompt Engineering and LLM Steerability. Unlike generic chatbot assistants that provide unconstrained conversation, Project ROAR enforces strict pedagogical scaffolding, continuous cognitive difficulty calibration, retrieval-grounded instruction, and multi-variable empirical assessment.

The system coordinates four specialized LLM agents overseen by a centralized Finite State Machine. Every interaction—from dynamic lesson synthesis to 3-part challenge formulation and semantic scoring—is grounded in a structured 36-node prerequisite graph.

2. Multi-Agent Framework & Specialized Roles

Agent Name	Primary Role & Responsibilities	Tools & Infrastructure Used
Tutor Orchestrator agents/orchestrator.py	Acts as the master finite state machine. Manages student state transitions (Onboarding → Lesson → Quiz → Evaluation → Node Passed), resolves prerequisite DAG unlocks, and persists session progress.	State Machine Controller, SQLite persistence, Activity Tracker.
Lesson Agent agents/lesson_agent.py	Queries domain documentation and synthesizes publication-grade study guides tailored to the learner's experience level, topic Bloom taxonomy, and continuous weight.	ChromaDB Vector Store (Semantic RAG), ModelManager LLM Router.
Quiz Agent agents/quiz_agent.py	Generates dynamic 3-part assessments calibrated to topic weight: (1) Theory MCQ, (2) Applied Prompt Construction, (3) Edge-case Refinement. Formulates scaffolded retry challenges.	ModelManager, JSON Schema Enforcer, Bloom Taxonomy Calibrator.
Hint Agent agents/quiz_agent.py	Provides progressive, non-repeating scaffolding across 3 levels: Level 1 (Mental Model), Level 2 (Structural Rules), Level 3 (Tactical Guidance). Tracks history to guarantee zero duplicates.	Deduplication Filter, Session History Memory, ModelManager.
Evaluator Agent agents/evaluator_agent.py	Grades multi-question submissions using a combined LLM semantic judge and deterministic rubric matcher. Calculates the 5-variable adaptive scoring formula with time and hint penalties.	LLM Semantic Judge, 5-Variable Scoring Engine, Rubric Matcher.

3. Tools, Technologies & Infrastructure

Component / Tool	Technology & Integration Details
Vector Store (RAG)	ChromaDB (rag/vector_store.py). Ingests and stores semantic vector embeddings of prompt engineering research papers, textbooks, and operational guidelines to ground generated lessons.
Inference Router	ModelManager (backend/model_manager.py). Unified inference abstraction supporting Ollama Cloud Web API (https://api.ollama.com), local Ollama daemon, and OpenRouter API with live health testing.
Relational Persistence	SQLite + SQLAlchemy ORM (db/storage.py, db/models.py). Stores user accounts, bcrypt password hashes, learner profiles, mastery maps, quiz attempts, and empirical research telemetry.
Live Telemetry Stream	ActivityTracker (backend/activity_tracker.py). Real-time ring buffer logging inference roundtrip latency (ms), token usage counts, active models, and agent actions to the UI drawer.
Frontend Architecture	Vanilla ES6 Modules & Custom Design System (ui/static/). Monospaced developer-tool design system utilizing Geist and JetBrains Mono typography, custom CSS tokens, and full responsive layout.

4. 5-Variable Adaptive Scoring Mathematical Formulation

Student submissions are evaluated using a multi-factor adaptive scoring equation balancing semantic correctness, structural compliance, time efficiency, hint dependency, and historical retry penalty:

Final Score Formula:
$$S_{\text{final}} = \max(0.0, (0.50 * S_{\text{semantic}} + 0.50 * S_{\text{rule}}) - P_{\text{hint}} - P_{\text{time}} - P_{\text{retry}})$$

- S_semantic (50%):** LLM Judge evaluation of theoretical depth, reasoning validity, and accuracy.
- S_rule (50%):** Deterministic keyword matching, structural delimiters, and length checks.
- P_hint:** Scaffolding penalty: $0.05 * \text{hints_used}$ (max 0.15 for 3 hints).
- P_time:** Overtime penalty: $\max(0.0, ((\text{time_elapsed} - \text{time_budget}) / \text{time_budget}) * 0.15)$.
- P_retry:** Consecutive fail streak penalty: $\min(0.15, \text{fail_streak} * 0.05)$.
- Passing Thresholds:** Tier 1 = 50%, Tier 2 = 60%, Tier 3 = 70%.

5. Complete File Base & Directory Map

File / Directory Path	Architectural Responsibility
main.py	FastAPI application entry point, CORS middleware, and static asset mount.
api/main.py	Router aggregator combining all sub-routers under /api/v1.
api/middleware.py	JWT Bearer token verification and user context dependency injection.
api/routes/auth.py	Endpoints for user registration, authentication, and /me profile fetching.
api/routes/session.py	Active session management and live activity telemetry stream endpoint.
api/routes/lesson.py	Generates and serves RAG-augmented study notes for curriculum nodes.
api/routes/quiz.py	3-part challenge generation, hint requests, submission evaluation, and node advance.
api/routes/progress.py	Knowledge graph prerequisite tree and learner mastery status endpoint.
api/routes/final_exam.py	3-phase standardized certification exam endpoint (Theory, Writing, Portfolio).
api/routes/settings.py	Model provider switching (Ollama Cloud/Local, OpenRouter) and live key validation.
agents/orchestrator.py	Master workflow state machine coordinator dispatching tasks to specialized agents.
agents/lesson_agent.py	Synthesizes structured, RAG-grounded study notes from vector search chunks.
agents/quiz_agent.py	Authors dynamic 3-part quizzes and generates progressive deduplicated hints.
agents/evaluator_agent.py	Evaluates student answers across semantic reasoning and rubric compliance.
core/curriculum.py	36-node knowledge graph ontology, Bloom levels, continuous weights, and prerequisites.
core/scoring.py	Implements the 5-variable adaptive scoring algorithm and rubric matching.
core/state_machine.py	Finite state machine transitions (Onboarding → Lesson → Quiz → Evaluation).
core/learner_profile.py	Learner mastery map, fail streak counters, and preferences management.
backend/model_manager.py	Multi-backend LLM inference router with live health check and fallback synthesizers.
backend/activity_tracker.py	Captures real-time telemetry events for streaming to the UI activity drawer.
rag/vector_store.py	ChromaDB vector collection management, document chunking, and semantic querying.
db/storage.py	SQLite database CRUD operations, user profile saving, and session lifecycle.
db/models.py	SQLAlchemy ORM database models and Pydantic request/response schemas.
data/curriculum_tree.json	Knowledge tree dataset with 36 nodes, difficulty tiers, and evaluation rubrics.
ui/static/css/app.css	Token-driven custom CSS system (Geist + JetBrains Mono, amber accent).
ui/static/index.html	Single-page application layout shell, brand mark, navigation tabs, and agent drawer.
ui/static/js/app.js	Frontend application controller, client router, and API fetch wrapper.
ui/static/js/views/	View renderers: lesson.js, quiz.js, score_card.js, login.js, etc.
tests/test_api.py	Automated integration test suite verifying auth, RAG, quizzes, and hints.

6. In-Depth Code Explanations & Implementation Logic

This section details the critical algorithms and architectural patterns powering Project ROAR's backend and agent coordination:

6.1 State Transition Controller & DAG Unlocking (`core/state_machine.py`)

Workflow: Student state is represented by the `TutorState` dataclass. When a user requests a lesson or submits answers, the `StateTransitionController` validates phase legality. Upon scoring $\geq S_{\text{pass}}$ (50%–70%), the node is recorded in `profile.completed_nodes`. The orchestrator queries `curriculum_graph.get_unlocked_nodes()`, filtering nodes whose prerequisites are fully satisfied, dynamically determining the next available topics.

6.2 Semantic Vector Search & RAG Synthesis (`agents/lesson_agent.py`)

Workflow: When `LessonAgent.generate_lesson_notes()` runs, it executes `vector_store.query_similar(query, n_results=3)` against ChromaDB using cosine distance over chunked research documents. The retrieved context is injected into an LLM prompt conditioned on the student's experience tier (Beginner vs. Expert) and Bloom taxonomy level, outputting structured Markdown with real worked examples and anti-patterns.

6.3 Dynamic 3-Part Challenge & Progressive Hint Deduplication (`agents/quiz_agent.py`)

Workflow: The Quiz Agent generates a structured JSON object containing three questions calibrated by weight. When the student clicks 'Hint', `generate_hint()` inspects `state.delivered_hints`, injects past hints into the LLM prompt with a strict anti-repetition directive, and checks for word-overlap (Jaccard similarity > 0.70). If a duplicate is detected, it falls back to a deterministic, level-indexed progressive hint (Level 1 Intuition → Level 2 Rules → Level 3 Tactical Guidance).

6.4 Evaluator Engine & Penalty Calculus (`core/scoring.py` & `evaluator_agent.py`)

Workflow: `EvaluatorAgent.evaluate_submission()` executes two evaluators in parallel: (1) An LLM judge evaluates semantic depth (S_{semantic}), and (2) deterministic regex functions inspect rubric key concepts and structural markers (S_{rule}). The scoring engine applies overtime penalties (P_{time}) if the duration exceeds $T = 180 + 60 \times \text{weight}$ seconds, scaffolding deductions ($P_{\text{hint}} = 0.05 \times \text{hints_used}$), and fail-streak penalties (P_{retry}).

6.5 Multi-Provider LLM Inference Router (`backend/model_manager.py`)

Workflow: `ModelManager.generate_async()` routes requests dynamically based on the active provider setting: Ollama Cloud Web API (<https://api.ollama.com/api/generate> with Bearer token authentication), local Ollama (11434), or OpenRouter (<https://openrouter.ai/api/v1/chat/completions>). It measures roundtrip latency via high-resolution timers, captures token usage counts, streams live telemetry to `ActivityTracker`, and activates dynamic fallback synthesizers if upstream providers experience network jitter.

6.6 Frontend SPA Architecture & Telemetry Stream (`ui/static/js/app.js`)

Workflow: Built without heavy bundlers or node runtime dependencies using pure ES6 modules and custom CSS design tokens. `app.js` manages client routing (Study, Progress Graph, Model Settings, Final Exam), injects Bearer tokens on all fetch calls, updates live countdown timers, and renders real-time agent activity feeds via SVG status indicators.

7. Quality Assurance, Testing & Research Telemetry

Every student attempt logs granular analytics into SQLite (`research_metrics` and `analytics_events`) to support academic research in adaptive learning trajectories. The entire system is continuously verified via automated integration tests: `pytest tests/test_api.py -v` (100% passing across authentication, RAG lessons, 3-part challenges, and progressive hints).